CO 487 Course Notes: Applied Cryptography

Winter 2026 - Samuel Jaques

Talha Yildirim,
tyildir [at] uwaterloo [dot] ca

Contents

1 Intro and History . . . 3
2 Symmetric Encryption . . . 6
2.1 Block Ciphers . . . 6
2.2 Block Cipher Modes of Operations . . . 14
3 Hash Functions . . . 18
3.1 Message Authentication Codes Authenticated Encryption . . . 25
3.2 Chosen Ciphertext Attacks . . . 29
3.3 Pseudorandom Functions . . . 29
3.4 Password Hashing . . . 31

1 Intro and History

Definition 1.1 (Cryptography)

Cryptography is about securing communications in the presence of malicious adversaries ♣

Remark

States of information

- Data at rest
- Data at transit
- Data while processing

Corollary 1.1.1 (Fundamental Goals of Cryptography)

- **Confidentiality** - Keeping data secret from all but those authorized to see it
- **Integrity** - Ensuring data has not been altered by unauthorized means
- **Authentication** - Corroborating the source of data or identity of an entity
- **Non-repudiation** - Preventing an entity from denying previous commitments or actions
- **Deniability** - Allowing an entity to deny previous commitments or actions
- **Consensus** - Ensuring a number of entities agree on the state of some data
- **Availability** - Ensuring a computer system works even if parts of it fail or are tampered with

♡

Remark

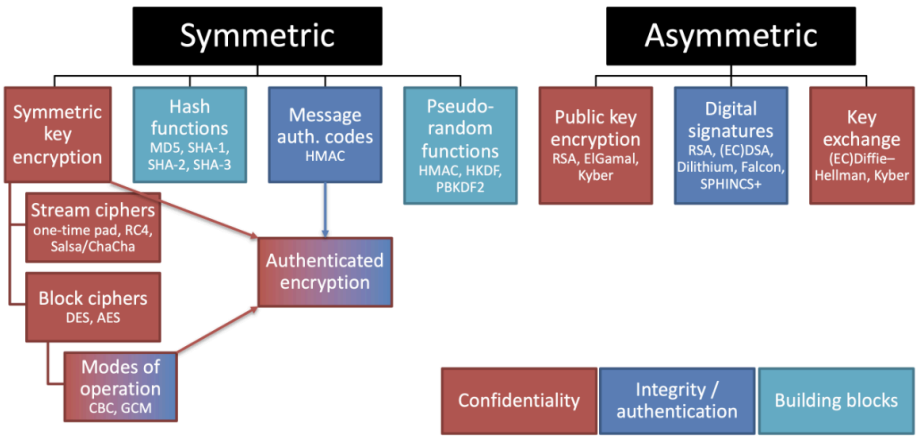


Figure 1: Cryptography building blocks

Symmetric cryptography: Uses a single shared secret to protect data. Both parties use the same secret, so it is simple and efficient, but requires the secret to be shared securely.

Asymmetric cryptography: Uses two related keys, one public and one private. This allows parties to communicate or verify identity without sharing a secret first, at the cost of being more complex and slower.

Definition 1.2 (Caesar Cipher)

Caesar cipher is a simple substitution cipher that encrypts text by shifting each letter a fixed number of positions forward in the alphabet, wrapping around at the end

Encrypt(m) where $m \in \{A, \ldots, Z\}^*$

```
for i = 1, ..., |m|
    x ← Encode(m_i)
    y ← x + 23 mod 26
    (or equivalently y ← x − 3 mod 26)
    c_i ← Decode(y)
return c
```

Decrypt(c) where $c \in \{A, \ldots, Z\}^*$

```
for i = 1, ..., |c|
    x ← Encode(c_i)
    y ← x − 23 mod 26
    (or equivalently y ← x + 3 mod 26)
    m_i ← Decode(y)
return m
```

Encode / Decode:

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

This scheme is **NOT** secure ♣

Remark

Note the difference between "encrypt" and "encode", and "decrypt" and "decode"

- **encode and decode** map letters to numbers without trying to add security – just mapping to a more convenient space
- **encrypt and decrypt** try to add security

Definition 1.3 (Shift Cipher)

Idea: Modify Caesar cipher by introducing a secret key

Key space

$\mathcal{K} = \{0, \ldots, 25\}$

Randomly sample key $k \leftarrow_{\$} \mathcal{K}$

Encrypt(k, m) where $m \in \{A, \ldots, Z\}^*$

```
for i = 1, ..., |m|
    x ← Encode(m_i)
    y ← x + k mod 26
    c_i ← Decode(y)
return c
```

Decrypt(k, c) where $c \in \{A, \ldots, Z\}^*$

```
for i = 1, ..., |c|
    x ← Encode(c_i)
    y ← x − k mod 26
    m_i ← Decode(y)
return m
```

This scheme is **NOT** secure ♣

How might we break the shift cipher?

Assumptions

Definition 1.4 (Kerckhoff's Principal)

We know the encryption / decryption algorithm being used but not any secret keys ♣

and,

We are given a cipher text to break

Two approaches

1. Try all 26 possible secret key values $k = 0, \ldots, 25$ (brute force / exhaustive key search)
2. Frequency Analysis

Definition 1.5 (Frequency Analysis)

Compare the distribution of letters in the cipher text with the distribution of letters in the underlying plain text space

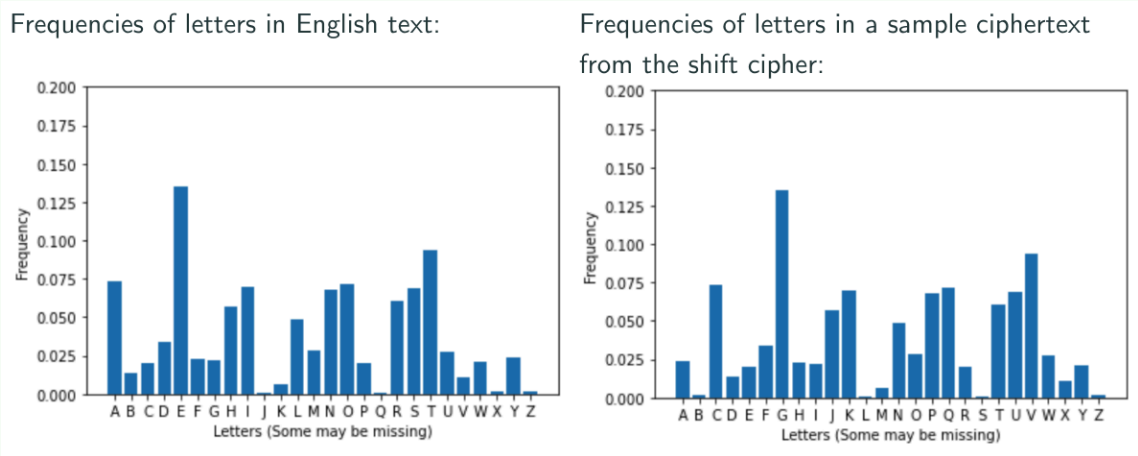


Figure 4: $k = 2$

♣

Definition 1.6 (Vigenere Cipher)

Use different shift ciphers for different parts of the message to reduce effect of frequency analysis

Key space	Message and ciphertext space
$\mathcal{K} = \{A, \ldots, Z\}^B$	$\mathcal{M} = \mathcal{C} = \cup_{i \geq 0}\{A, \ldots, Z\}^{iB}$

Encrypt(k, m)

```
for i = 1, ..., |m|
    c_i ← m_i + k[i mod B] mod 26
return c
```

Decrypt(k, c)

```
for i = 1, ..., |c|
    m_i ← c_i − k[i mod B] mod 26
return m
```

Example with block length $B = 6$

$m =$	t	h	i	s	i	s	a	m	e	s	s	a	g	e
$+\, k =$	C	R	Y	P	T	O	C	R	Y	P	T	O	C	R
$c =$	V	Y	G	H	B	G	C	D	C	H	L	O	I	V

This scheme is **NOT** secure

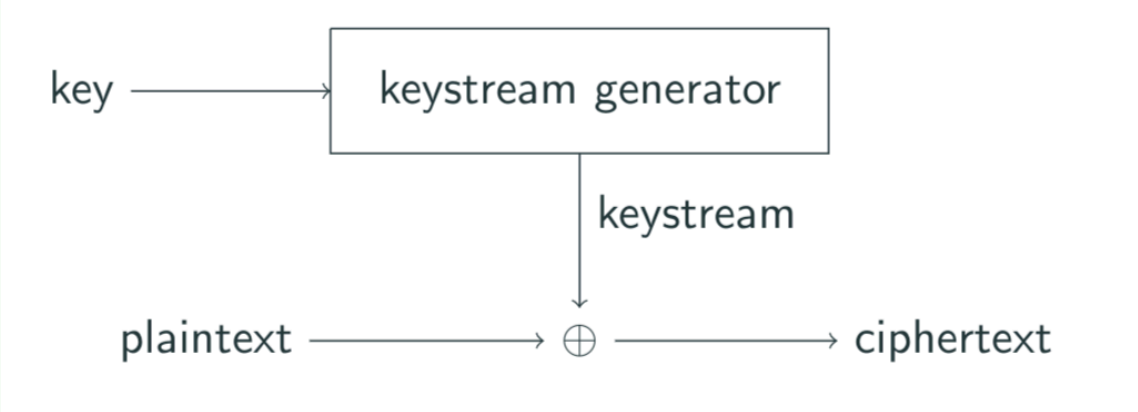
♣

2 Symmetric Encryption

2.1 Block Ciphers

Definition 2.1.1 (Stream Cipher)

A stream cipher is a symmetric key encryption scheme in which each successive character of plaintext determines a single character of ciphertext

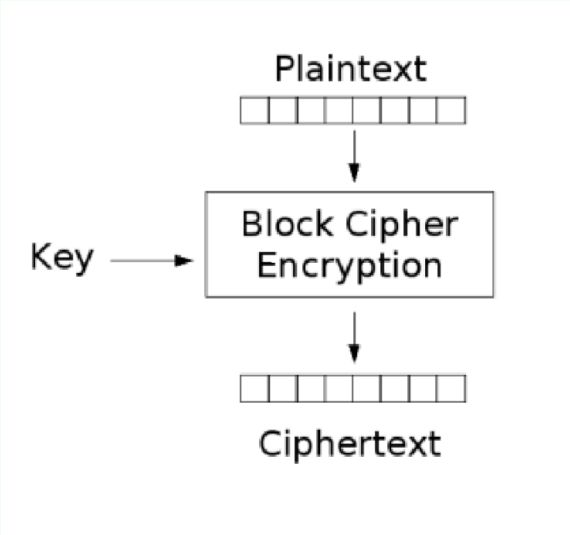


♣

Definition 2.1.2 (Block Cipher)

A block cipher is a symmetric key encryption scheme in which a fixed length block of plaintext determines an equal sized block of ciphertext

e.g. AES, DES

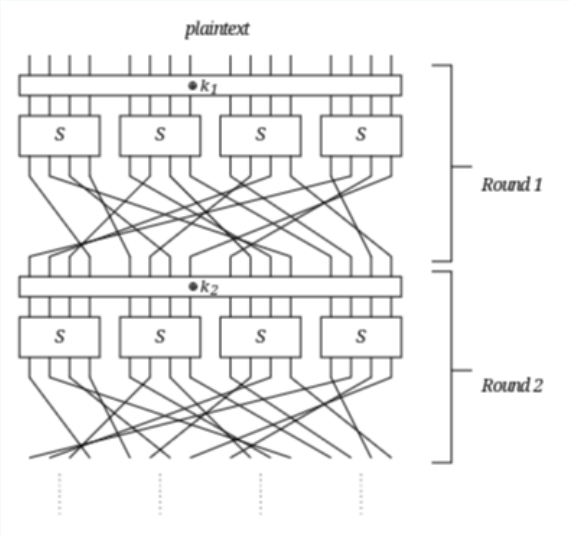


Corollary 2.1.2.1 (Desirable Properties of Block Ciphers)

- **Security**
  - **Diffusion:** Each ciphertext bit should depend on all plaintext and all key bits
  - **Confusion:** The relationship between key bits, plaintext bits, and ciphertext bits should be complicated
  - **Cascade or Avalanche Effect:** Changing one bit of plaintext or key should change each bit of ciphertext with 50% probability
  - **Key Length:** Should be small, but large enough to preclude exhaustive key search
- **Efficiency**
  - Simplicity (easier to implement and analyze)
  - High encryption and decryption rate
  - Suitability for hardware or software

Definition 2.1.3 (Substitution Permutation Networks)

A substitution permutation network (SPN) is a multiple round iterated block cipher where each round consist of a substitution operation followed by a permutation operation. During each round, a round key is `XORed` into the state. The round keys k_i are derived from the main key k using a key schedule function.



♣

Warning

Despite being called a Substitution Permutation Network, the permutation can be any invertible linear function.

Remark

To a mathematician, any bijective (one-to-one) function on a finite set can be considered a "permutation". To avoid confusion lets clarify permutation and substitution in a substitution permutation network.

1. **Bijection or Substitution**

The S-box will (often) be a one-to-one function on some small space

e.g. 4 or 8 bit strings

11110001 $\rightarrow$ 10100001

11110010 $\rightarrow$ 01000101

In this course this will be referred to as a bijection or substitution

2. **Permutation**

The permutation will be a one-to-one function of positions of bits.

e.g. A block of 128 bits

$p : \{0, 1, 2, ..., 127\} \rightarrow \{0, 1, 2, ..., 127\}$

This tells us to move the bit at position i to position $p(i)$

Remark

Two perspectives on a permutation

1. As a function from bit position to bit position (i.e a one-to-one function on $\{0, 2, \ldots\}, 127$)
2. As a function from bit strings to bit strings (i.e take each bit in the bit string and rearrange it according to how the positions are permuted)

As an example, we could have a permutation like this:

Start Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
End Position	11	13	15	0	8	7	9	14	5	2	10	12	6	1	4	3

which would act on 16-bit strings as dictated above, which could also be done by a matrix:

$$\begin{pmatrix}
0&0&0&1&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&1&0&0&0\\
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&1\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&1&0\\
0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&1&0&0&0\\
0&0&0&0&0&1&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&1&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&1&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&1&0&0&0&0&0\\
1&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&1&0&0&0&0\\
0&1&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&0\\
0&0&1&0&0&0&0&0&0&0&0&0&0&0&0&0
\end{pmatrix}
\begin{pmatrix}0\\1\\1\\0\\0\\0\\0\\1\\1\\1\\1\\0\\0\\1\\0\\1\end{pmatrix}
=
\begin{pmatrix}0\\1\\1\\1\\0\\1\\0\\0\\0\\1\\0\\0\\1\\1\\0\\1\end{pmatrix}$$

Definition 2.1.4 (The Advanced Encryption Standard (AES))

Requirements

- **Key Sizes:** 128, 192 and 256 bits
- **Block Sizes:** 128 bits
- Efficient on both hardware and software platforms
- Availability on a worldwide, non-exclusive, royalty-free basis
- AES is a SPN where the "permutation" operation consist of two linear transformations (one of which is a permutation)
- All operations are byte oriented

Number of rounds depends on the key length

key length	number of rounds h
128	10
192	12
256	14

- The substitution operation (S-box) is the only non-linear component
- The permutation operations (permutation and linear transformations) spread out the non-linearities in each round

♣

Corollary 2.1.4.1 (AES Round Operations)

- Each round updates a variable called `State` which consist of a $4x4$ array of bytes (notes: $4 \times 4 \times 8 = 128$, the block size)
- `State` is initialized with the 128-bit plaintext

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$

$\longleftarrow$ *plaintext*

- After h rounds are completed, one final additional round key is `XORed` with `State` to produce the ciphertext
- The AES round function uses four operations:
  - `AddRoundkey` , key mixing
  - `SubBytes` , S-box
  - `ShiftRows` , permutation
  - `MixColumns` , matrix multiplication / linear transformation

Corollary 2.1.4.2 (AES Encryption)

From the key k, derive $h+1$ round keys $k_0, k_1, \ldots, k_h$ via the key schedule

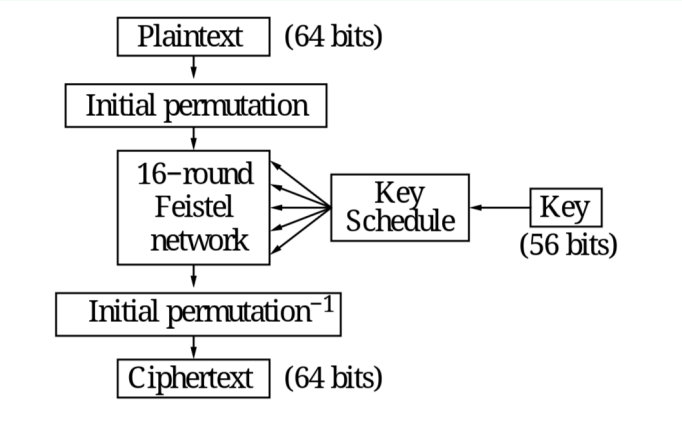
Encryption function:

$$
\begin{aligned}
&\text{State} \leftarrow \text{plaintext} \\
&\text{for } i = 1 \ldots h-1 \text{ do} \\
&\qquad \text{State} \leftarrow \text{State} \oplus k_{i-1} \\
&\qquad \text{State} \leftarrow \texttt{SubBytes}(\text{State}) \\
&\qquad \text{State} \leftarrow \texttt{ShiftRows}(\text{State}) \\
&\qquad \text{State} \leftarrow \texttt{MixColumns}(\text{State}) \\
&\text{State} \leftarrow \text{State} \oplus k_{h-1} \\
&\text{State} \leftarrow \texttt{SubBytes}(\text{State}) \\
&\text{State} \leftarrow \texttt{ShiftRows}(\text{State}) \\
&\text{State} \leftarrow \text{State} \oplus k_h \\
&\text{ciphertext} \leftarrow \text{State}
\end{aligned}
$$

Not in the final round `MixColumns` is not applied

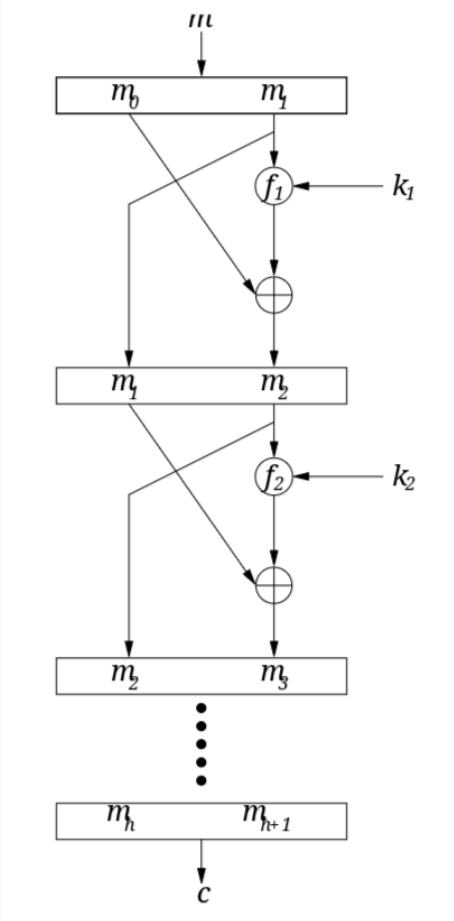
Definition 2.1.5 (Data Encryption Standard DES)

Block cipher with 64-bit blocks, 56-bit key, and 16 rounds of operation



Definition 2.1.6 (Feistel Network Design)

- DES uses a Feistel network design
- Plaintext is divided into two halves
- Key is used to generate subkeys $k_0, k_1, \ldots, k_h$
- f_i is a component function whose output value depends on k_i and m_i



Corollary 2.1.6.1 (Feistel Ciphers: A Class of Block Ciphers)

- **Components of a Feistel Cipher:**
  - Parameters: n (half the block length), h (number of rounds). l (key size)
  - $M = \{0,1\}^{2n}, C = \{0,1\}^{2n}, K = \{0,1\}^l$
  - A key scheduling algorithm which determines subkeys $k_0, k_1, \ldots, k_h$ from a key k
  - Each subkey k_i constructs a component function $f_i : \{0,1\}^l \times \{0,1\}^n \to \{0,1\}^n$
- **Encryption takes h rounds:**
  - Plaintext is $m = (m_0, m_1)$, where $m_i \in \{0,1\}^n$
  - **Round 1:** $(m_0, m_1) \to (m_1, m_2)$, where $m_2 = m_0 \oplus f_1(k_1, m_1)$
  - **Round 2:** $(m_1, m_2) \to (m_2, m_3)$, where $m_3 = m_1 \oplus f_2(k_2, m_2)$
  - **Round h:** $(m_{h-1}, m_h) \to (m_h, m_{h+1})$, where $m_{h+1} = m_{h-1} \oplus f_h(k_h, m_h)$
  - Ciphertext is $c = (m_h, m_{h+1})$
- **Decryption:** Given $c = (m_h, m_{h+1})$ and k, to find $m = (m_0, m_1)$
  - Compute $m_{h-1} = m_{h+1} \oplus f_h(k_h, m_h)$
  - Similarly, compute $m_{h-2}, \ldots, m_1, m_0$

♡

Remark

- No restrictions on the functions f_i in order for the encryption procedure to be invertible
- **Underlying principle:** Take something "simple" and use it several times; hope that the result is "complicated"

Corollary 2.1.6.2 (DES Problem: Small Key Size)

Exhaustive seach on key space takes 2^{56} steps and can be easily parallelized

♡

Definition 2.1.7 (Multiple Encryption)

Re-encrypt the ciphertext one or more times using independent keys, and hope that this operation increase the effective key length

Multiple encryption does not always increase security. e.g. Substitution cipher

♣

Definition 2.1.8 (Double Encryption)

- **Double DES:** Key is $k = (k_1, k_2), k_1, k_2 \underset{R}{\in} \{0,1\}^{56}$
- **Encryption:** $c = E_{k2}(E_{k1}(m))$
- **Decryption:** $c = E_{k1}^{-1}(E_{k2}^{-1}(m))$
- Key length of Double DES is $l = 112$, so exhaustive key search takes 2^{112} steps (infeasible)

♣

Corollary 2.1.8.1 (Attack on Double DES)

Main idea: If $c = E_{k2}(E_{k1}(m))$, then $E_{k2}^{-1}(c) = E_{k1}(m)$ (meet-in-the-middle)

1. Given: Known plaintext pairs (m_i, c_i) , $i = 1, 2, 3, \ldots$
2. For each $h_2 \in \{0,1\}^{56}$:
   - Compute E_{h2}^{-1}, and store $[E_{h2}^{-1}, h_2$ in a table
3. For each $h_1 \in \{0,1\}^{56}$ do the following
   - Compute $E_{h_1}(m_1)$
   - Search for $E_{h_1}(m_1)$ in the table
   - If $E_{h_1}(m_1) = E_{h_2}^{-1}(c_1)$:
     - Check if $E_{h_1}(m_1) = E_{h_2}^{-1}(c_2)$
     - Check if $E_{h_1}(m_1) = E_{h_2}^{-1}(c_3)$
     - Etc
     - If all checks pass, then output (h_1, h_2) and stop

Complexity of the attack is $\approx 2^{57}$

♡

Remark

- Number of known plaintext / ciphertext pairs required to avoid false keys: 2 suffice, with high probability
- Number of DES operations $\approx 2^{56} + 2^{56} + 2 \times 2^{48} \approx 2^{57}$
- Space requirements: $2^{56}(64 + 65)$ bits $\approx 1,080,863$ Tbytes

Double DES effectively has the same key length as DES and isn't much more secure

Definition 2.1.9 (Three Key Triple Encryption)

- **Triple DES:** Key is $k = (k_1, k_2, k_3), k_1, k_2, k_3 \underset{R}{\in} \{0,1\}^{56}$
- **Encryption:** $c = E_{k3}(E_{k2}(E_{k1}(m)))$
- **Decryption:** $c = E_{k1}^{-1}\left(E_{k2}^{-1}\left(E_{k_3}^{-1}(m)\right)\right)$
- Key length of Triple DES is $l = 168$, so exhaustive key search takes 2^{168} steps (infeasible)

♣

Corollary 2.1.9.1 (Attack on Triple DES)

- meet-in-the-middle attack takes $\approx 2^{112}$ steps
- So, the effective key length of Triple DES against exhaustive key search is ≤ 112 bits
- No proof that Triple DES is more secure than DES
- Block length is 64 bits, and now forms the weak link:
  - Adversary stores a large table (of size $\leq 2^{64}$) pf (m, c) pairs (dictionary attack)
  - To prevent this attack $\rightarrow$ change secret keys frequently

♡

2.2 Block Cipher Modes of Operations

Remark

Recall,

Stream Cipher: A symmetric key encryption scheme in which each a pseudorandom sequence of arbitrary length is generated to encrypt successive character of plaintext of ciphertext

Block Cipher: A symmetric key encryption scheme in which a fixed length block of plaintext determines an equal sized block of ciphertext

Lemma 2.2.1 (Encrypting Bulk Data)

What if one needs to encrypt large quantities of data?

- With a stream cipher, just encrypt each character
- With a block cipher, there are some complications if:
  - The input is larger than one block
  - The input does not fill an integer number of blocks

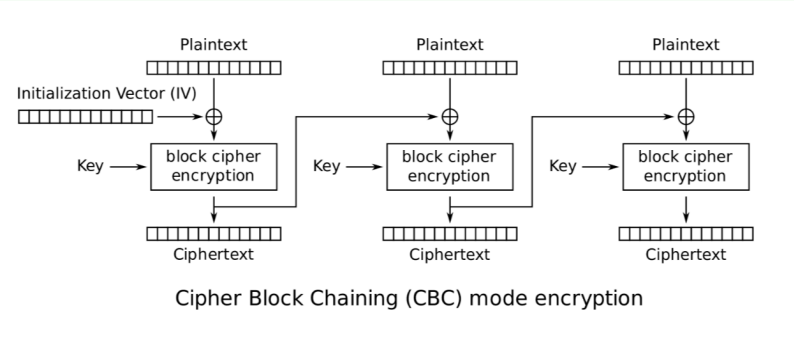
To deal with these problems we use a mode of operation, which means a specification for how to encrypt multiple and / or partial data blocks using a block cipher

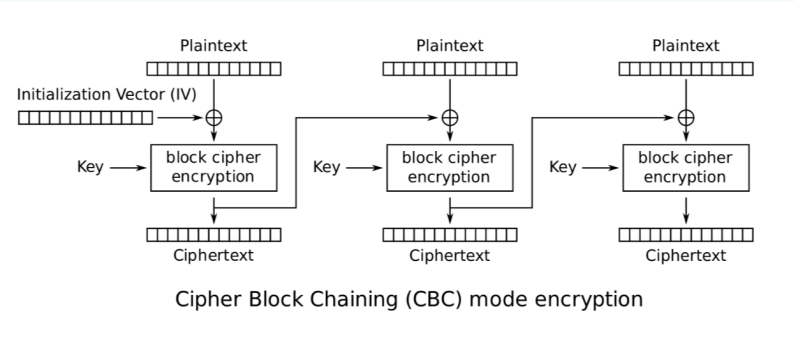
Definition 2.2.2 (Padding)

- Some modes namely ECB and CBC, require the plaintext to consist of one or more complete blocks
- Padding method
  - Append a single '1' bit to the data string
  - Pad the resulting string by as few '0' bits, possible none, as are necessary to complete the final block
- The padding bits can be removed unambiguously, if the receiver known that this padding method is used

Definition 2.2.3 (Cipher Block Chaining mode (CBC))

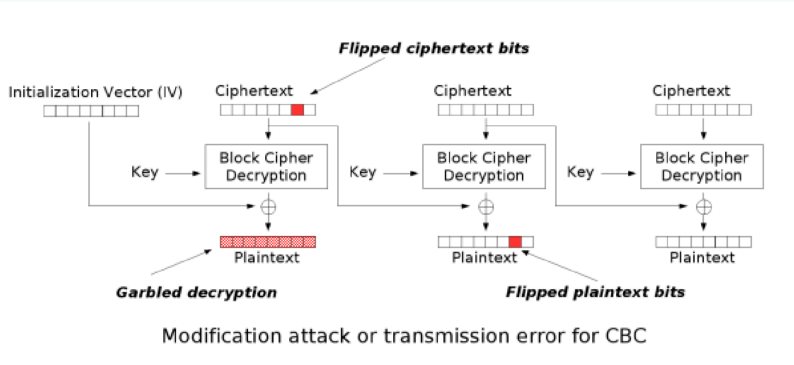
Use a single initialization vector for the first block of plaintext, and "chain" the (pseudorandom) ciphertext blocks as the next blocks initialization vectors. The IV is included as part of the ciphertext





- **Encryption:**
  - $C_i = E(K, P_i \vee C_{i-1})$, where $C_0 = IV$
  - P_i is `XORed` with the previous ciphertext block C_{i-1}, and encrypted with the key K to produce ciphertext block C_i. For the first plaintext block IV is used for the value of C_0
- **Decryption:**
  - $P_i = D(K, C_i) \oplus C_{i-1}$
  - C_i is decrypted with the K ,and `XORed` with the previous ciphertext block C_{i-1} to produce plaintext block P_i. As in encyption, IV is used in place of C_0

CBC Mode Error Propagation



♣

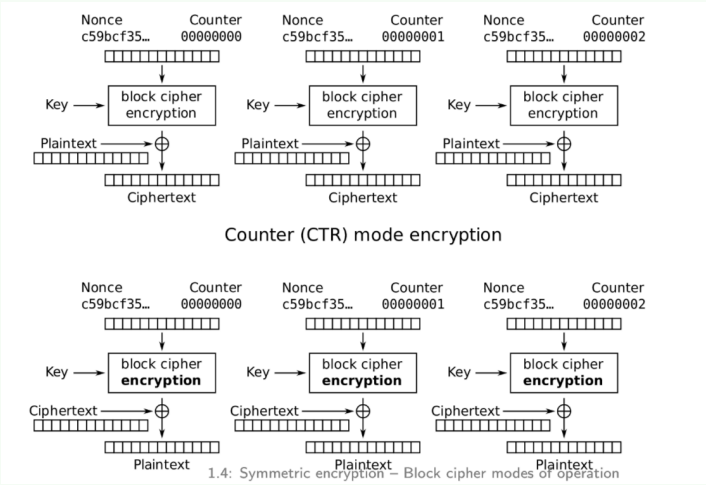
Corollary 2.2.3.1 (CBC Mode Properties)

Randomised	✓
Padding	Required
Error propagation	Errors propagate within blocks and into specific bits of next block
Dropped blocks	Can't decrypt next block, can decrypt subsequent blocks
IV	Must be random
Parallel encryption?	✘
Parallel decryption?	✓
IND-CPA secure?	✓

CBC mode is commonly used for bulk encryption and is supported in most libraries and protocols

Definition 2.2.4 (Counter Mode (CTR))

Choose a nonce at random during encryption. Prepend the nonce to the ciphertext



CTR is a synchronous stream cipher. The keystream is generated by encrypting successive values of a "counter", initialized using a nonce (randomly chosen value) N:

$$O_i = E(K, T_i)$$

Where $T_i = N \parallel i$ is the concatenation of the nonce and block number i

- **Encryption:**
  - $C_i = O_i \oplus P_i$
- **Decryption:**
  - $P_i = O_i \oplus C_i$
- One bit change in ciphertext produces a one bit change in the plaintext at the same location

Corollary 2.2.4.1 (CTR Mode Properties)

The underlying block cipher is only used in encryption mode.

Randomised	✓
Padding	Required
Error propagation	Errors propagate within blocks and into specific bits of next block
Dropped blocks	Can't decrypt next block, can decrypt subsequent blocks
IV	Must be random
Parallel encryption?	✘
Parallel decryption?	✓
IND-CPA secure?	✓

- CTR mode is a synchronous stream cipher mode
- CTR mode is also good for access to specific plaintext blocks without decrypting the whole stream

Remark

Summary of Mode Properties

	ECB	CBC	CFB	OFB	CTR
IND-CPA secure?	✘	✓	✓	✓	✓
Randomised	✘	✓	✓	✓	✓
Padding required?	✓	✓	✘	✘	✘
Error propagation within block? next? subsequent?	✓✘✘	✓✓✘	✓✓✘	✓✘✘	✓✘✘
Dropped block decrypt next? subsequent?	✓✓	✘✓	✘✓	✓* ✓*	✓* ✓*
IV	none	random	unique	unique	unique
Parallel encryption?	✓	✘	✘	✘	✓
Parallel decryption?	✓	✓	✓	✘	✓
Pre-compute encryption?	✘	✘	✘	✓	✓
Pre-compute decryption?	✘	✘	✘	✓	✓

* if receiver realizes bock has been dropped and advances appropriately

3 Hash Functions

Definition 3.1 (Hash Function)

- A hash function is a checksum designed to be sage from malicious tampering
- H is called an n-bit hash function
- $H(x)$ is called the hash value, hash, or message digest of x
- Description of a hash function is public. There are no secret keys.

A hash function is a mapping H such that:

1. H maps inputs of arbitrary lengths to outputs of length n, where n is fixed ($H : \{0,1\}* \to \{o,1\}^n$) (more generally, H maps elements of a set S to a set T where $|S| > |T|$)
2. $H(x)$ can be efficiently computed for all $x \in \{0,1\}*$

♣

Corollary 3.1.1 (Typical Cryptographic Requirements (Informally))

- **Pre-image Resistance:** Hard to invert given just an output
- **2nd Pre-image Resistance:** Hard to find a second input that has the same hash value as a given first input
- **Collision Resistance:** Hard to find two different inputs that have the same hash values

♡

Definition 3.2 (Pre-image Resistance)

Let m be a positive integer. We say that H is *pre-image* resistant for messages of length m if, given $y = H(x)$ for x picked uniformly at random from $\{0,1\}^m$, it is computationally infeasible to find (with non-negligible success probability) any input z such that $H(z) = y$

- z is called a pre-image of y

♣

Definition 3.3 (Second Pre-image Resistance)

Let m be a positive integer. We say that H is second pre-image resistant for messages of length m if, given an input $x \underset{R}{\in} \{0,1\}^m$, it is computationally infeasible (with non-negligible success probability) to find a second input $x' \neq x$ such that $H(x) = H(x')$

- $x \underset{R}{\in} \{0,1\}^m$ means x chosen uniformly at random from $\{0,1\}^m$

♣

Definition 3.4 (Collision Resistance)

It is computationally infeasible (with non-negligible success probability) to find two distinct input x, x' such that $H(x) = H(x')$

- The pair (x, x') is called a collision for H

♣

Remark

How are second pre-image resistance and collision resistance different?

- **Collision Resistance:** The attacker has freedom to pick both x and x'
- **Second Pre-image Resistance:** The attacker given some x and has to find an x'

Definition 3.5 (One Way Hash Function)

A has function that is pre-image resistant is sometimes called a one way hash function (OWHF)

♣

Definition 3.6 (Cryptographic Hash Function)

A hash function that is pre-image, second pre-image, and collision resistant is called a cryptographic has function

♣

Some Applications of Hash Functions

- **Password protection on a multi user computer system**
- **Modification Detection Codes (MDC)**
  - Ensure that a message m is not modified by unauthorized means, one computes $H(m)$ and protects $H(m)$ for unauthorized modification
- **Message Digest for Digital Signature Schemes**
  - For reasons of efficiency, instead of signing a (long) message, the (much shorter) message digest is signed
- **Message Authentication Codes (MACs)**
  - Provides data integrity and data origin authentication
- **Pseudorandom Bit generation**
  - Distilling random bits s from several "random" sources $x_1, x_2, \ldots, x_t$
- **Key Derivation Function (KDF)**
  - Deriving a cryptographic key from a shared secret

Remark

Collision resistance is not always necessary

Corollary 3.6.1 (Popular Hash Functions)

- MD5
- SHA-1
- SHA-2
- SHA-3

Less widely used: MD2, MD4, Tiger

Not cryptographic hash functions: CRC8, CRC16, CRC32

♡

Tip

In cryptography we often want to show theorems like:

Theorem 3.7

If scheme S satisfies security property X, then scheme T (built using S) satisfies security property Y

Or,

Theorem 3.8

If there is no efficient adversary that breaks security property X of scheme S, then there is no efficient adversary that breaks security property Y of scheme T

It is often more convenient to prove the (equivalent) **contrapositive statement**:

Theorem 3.9 (Contrapositive Statement)

Suppose there exists an efficient adversary B that breaks security property Y of scheme T.

Then there exists an efficient adversary A (that uses B) that breaks security property X of scheme S

To prove the contrapositive statement, our task is coming up with the algorithm A, assuming that algorithm B exists with desired property

We can take this and do $\rightarrow$

Remark

Theorem 3.10

Collision resistance implies 2nd pre-image resistance

Or,

Theorem 3.11

If H is collision resistant, then H is 2nd pre-image resistant

Written in the contrapositive form:

Theorem 3.12

If there exists an efficient algorithm B that breaks the 2nd pre-image resistance of H, then there exists an efficient algorithm A that breaks the collision resistance of H

Tip

In cryptography we sometimes want to show theorems like the following:

Theorem 3.13

Security property X does not imply security property Y

Another way of phrasing this is:

Theorem 3.14

There exists a scheme S that has security property X but does not have security property Y

Often we prove this by taking a generic scheme S that has property X, then construct an "artificially degenerate" modified scheme S^* that still has property X, but clearly doesn't have property Y

Definition 3.15 (Generic Attacks)

- A generic attack on a has function $H : \{0,1\}^* \to \{0,1\}^n$ does not exploit any properties a specific hash function may have
- In the analysis of a generic attack, we view H as a random function the sense that for each $x \in \{0,1\}^*$, the value $y = H(x)$ was chosen by selecting y uniformly at random from $\{0,1\}^n$ (written $y \underset{R}{\in} \{0,1\}^n$)
- From a security point of view, a random function is an ideal hash function. However, random functions are not suitable for practical applications because they cannot be compactly stored

Definition 3.16 (Generic Attack for Finding Pre-images)

Given $y \in \{0,1\}^n$, repeatedly select distinct $x' \in \{0,1\}^*$ until $H(x') = y$

- Expected number of steps is $\approx 2^n$ (each step is a hash function evaluation)
- This attack is infeasible if $n \Rightarrow 128$
- It is proven that this generic attack for finding pre-images is optimal

Definition 3.17 (Generic Attack for Finding Collisions)

Repeatedly select arbitrary distinct $x \in \{0,1\}^*$ and store $(H(x), x)$ in a table sorted by first entry. Continue until a collision is found

- Expected number of steps is $\sqrt{\frac{\pi 2^n}{2}} \approx \sqrt{2^n}$ (birthday paradox) (each step is a hash function evaluation)
- It is proven that this generic attack for finding collisions is optimal
- Expected space required: $\sqrt{\frac{\pi 2^n}{2}} \approx \sqrt{2^n}$
- This attack is infeasible if $n \geq 160$
- If $n = 128$
  - Expected running time: 2^{64} steps (barely feasible)
  - Expected space required: 7×10^8 Tbytes

♣

Theorem 3.18 (Birthday Paradox)

Suppose q values are picked uniformly at random, with replacement, from a set of size N. Then the probability of no collisions among the q selected values is

$$\text{Prob(no collision)} = \prod_{i=1}^{q-1}\left(1 - \frac{i}{N}\right)$$

Approximation used and generally good enough for class,

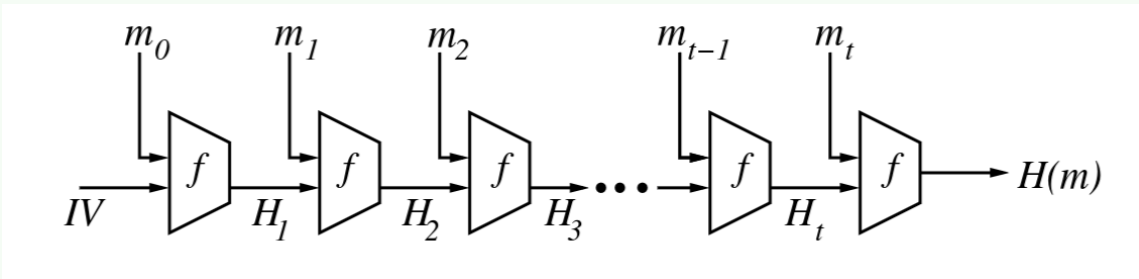
$$\text{Prob(collision)} \approx \frac{q^2}{N}$$

♡

Definition 3.19 (The Merkle Damgard Construction)

To compute $H(m)$ where m has bit length $b < 2^r$ do:

1. Break up m into r-bit blocks: $m^* = m_0, m_1, \ldots, m_{t-1}$, padding out the last block with 0 bits if necessary
2. Define m_t, the length block, to hold the right justified binary representation of b
3. Define $H_0 = IV$
4. Compute $H_{i+1} = f(H_i, m_i)$ for $i = 0, 1, 2, \ldots, t$
5. H_i's are called chaining variables
6. return $H(m) = H_{t+1}$



♣

Theorem 3.20 (Collision Resistance of Merkle Damgard)

If the compression function f is collision resistant, then the hash function H is also collision resistant

Definition 3.21 (SHA-2)

Description of SHA-256 (For SHA-384 and SHA-512, increase n to 512 and r to 1024):

- Iterated hash function (Merkle Damgard construction)
- $n = 256$, $r = 512$
- Compression function is $f : \{0,1\}^{256+512} \to \{0,1\}^{256}$
- Input: Bit string x of arbitrary bit length $b \geq 0$
- Output: 256-bit hash value $H(x)$ of x

Corollary 3.21.1 (Wangs Collision Finding Attack (SHA-1))

- Fix any n-bit string I
- Wangs attack finds two (different) 1 block messages $x = x_1$ and $y = y_1$ such that $F(I, x_1) = F(I, y_1)$
- The attack gives limited but not complete control over x_1 and y_1
- The attack takes about 2^{63} steps (versus usual 2^{80})
- By selecting $I = IV$ (where IV is the fixed initialization vector specified in SHA-1), Wang's attack can be used to find two one block messages x and y, so these messages are essentially meaningless

Lemma 3.22 (How to Exploit Single Hash Collision)

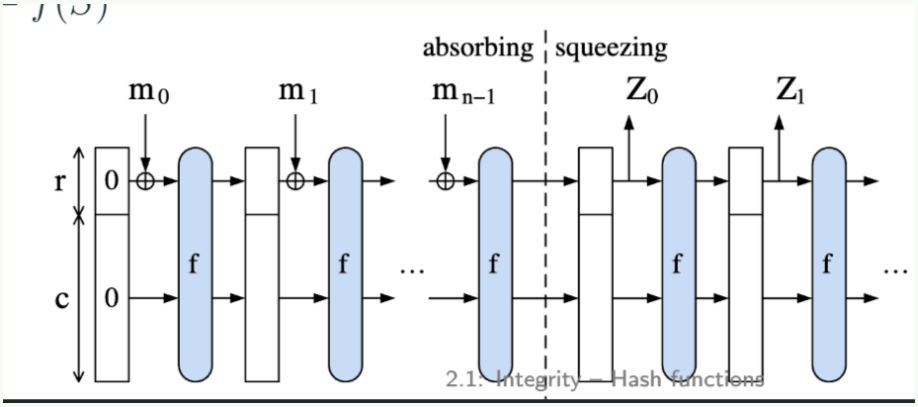
- Let H be a hash function
- Suppose that we can find two message x and y so that $H(x) = H(y)$
- Suppose that the collision finding method we use does not allow us to control the structure of x and y, so that these messages are essentially meaningless
- Suppose also that the collision finding methods take considerable (but feasible) time

Definition 3.23 (Sponge Function)

- **State:** $S \in \{0,1\}^b$
- **Function f:** $f : \{0,1\}^b \to \{0,1\}^b$, often a permutation
- A padding function
- The state S is divided into two parts: $R \in \{0,1\}^r$ and $C \in \{0,1\}^c$ where $b = r + c$; r is called the rate and c is the capacity

To hash a message m:

1. Initialize state $S = R \parallel C$ to zero
2. Pad message to break into r-bit blocks
3. [Absorb Stage] For each r-bit block B:
   - Replace $R \leftarrow R \oplus B$
   - Replace $S \leftarrow f(S)$
4. [Squeeze Stage] While more outputs bits are needed:
   - Output R
   - Replace $S \leftarrow f(s)$



♣

3.1 Message Authentication Codes Authenticated Encryption

Definition 3.1.1 (Message Authentication Code (MAC))

A message authentication code (MAC) scheme is an efficiently computable function

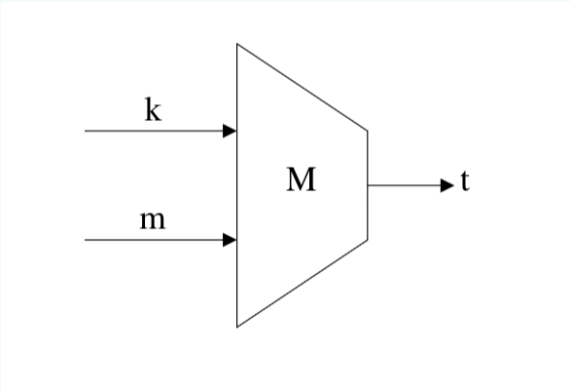
$$M : \{0,1\}^l \times \{0,1\}^* \to \{0,1\}^n$$

written

$$M(k,m) = t$$

Where k is the key, m is the message, and t is the tag

> MAC schemes are used for providing (symmetric key) data integrity and data origin authentication

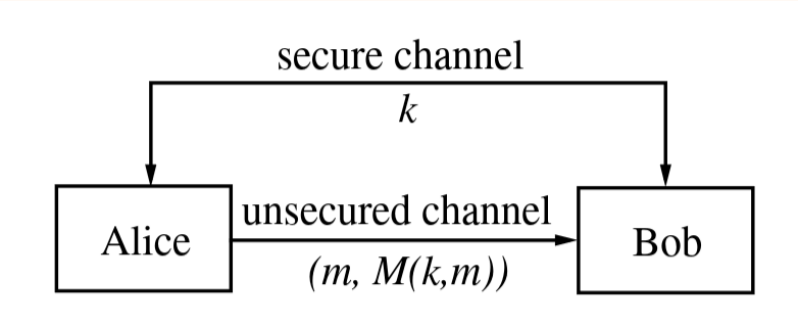


Corollary 3.1.1.1 (Applications of Message Authentication Codes)

1. Alice and Bob establish a secret key $k \in \{0,1\}^l$
2. Alice computes $t = M(k,m)$ and sends (m,t) to Bob
3. Bob verifies that $t = M(k,m)$

To avoid replay, add a timestamp, or sequence number

No confidentiality or non-repudiation



Let k be the secret key shared by Alice and Bob

The adversary knowns everything about the MAC scheme expect the value of k

Definition 3.1.2 (MAC security)

A MAC scheme is secure if:

- Given some number of MAC tags $M(k, m_i)$ for messages m_i chosen adaptively by the adversary (interaction)
- It is computationally infeasible (computational resources)
- To compute (with non non-negligible probability of success) the value of $M(k, m)$ for any message $m \neq m_i$ (goal)

In other words, a MAC scheme is secure if it is existentially unforgeable against chosen message attack

♣

Definition 3.1.3 (Generic Attacks)

Guessing the MAC of a message m:

- Select $y \in \{0, 1\}^n$ and guess that $M(k, m) = y$
- Assuming that $M(k, .)$ is a random function, the probability of success is $\frac{1}{2^n}$
- Guesses cannot be directly checked without interaction
- Depending on the application where the MAC algorithm is employed, one could choose n as small as 32. In general, $n \Rightarrow 128$ is preferred

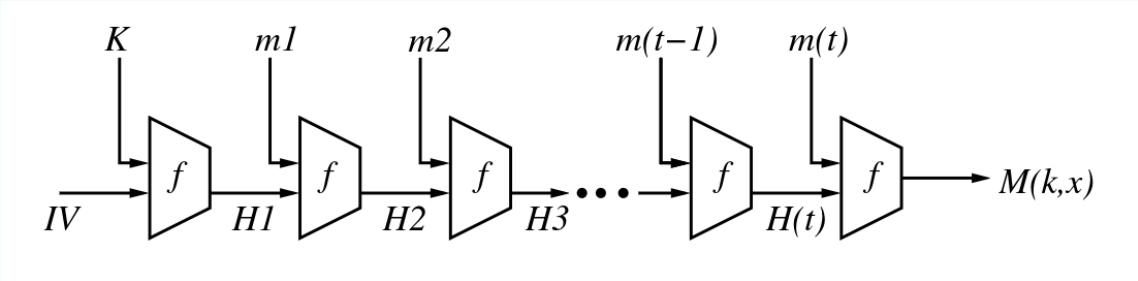
Exhaustive Search on the Key Space:

- Given r known message-MAC pairs: $(m_1, t_1), ..., (m_r, t_r)$ one can check whether a guess k of the key is correct by verifying that $M(k, m_i) = t_i$ for $i = 1, 2, ..., r$
- Assuming that the $M(k, .)$'s are random functions, the expected number of keys for which the tags verify is $K + \frac{2^l}{2^{nr}}$
- Requires $\approx 2^l$ computations
- Exhaustive search is feasible if $l \geq 128$

♣

Definition 3.1.4 (MACs based on has functions: Secret Prefix method)

MAC definition: $M(k, m) = H(K \parallel m)$



This is insecure. Here is a length extension attack:

- Suppose that $(m, M(k, m))$ is known
- Suppose that the bit length of m is a multiple of r
- Then $M(k, m \parallel m')$ can be computed for any m' (without knowledge of k)

Also insecure if a length block is post pended to $K \| m$ prior to application of H

♣

Definition 3.1.5 (Encrypt and MAC (E&M))

Consider encrypt and MAC

$$\text{Compute} \quad c = Enc(m) \quad \text{and} \quad t = MAC(m), \ \text{transmit} \quad c \parallel t$$

When decrypting, the recipient checks that the MAC is correct

- Problem: MAC's are not required to ensure confidentiality
  - For example, the MAC might leak one plaintext bit, and still be "secure" as a MAC as. Violates semantic security!
- Even if the SKES is secure and the MAC is secure, the encrypt and MAC combination might be insecure

♣

Definition 3.1.6 (MAC then encrypt (MtE)

Consider the MAC then encrypt

$$\text{Compute} \quad t = MAC(m) \quad \text{and} \quad c = Enc(m \parallel t), \ \text{transmist} \quad c$$

When decrypting, the recipient checks that the MAC is correct

- Problem: SKES's are not required to integrity
  - For example, changing the ciphertext might not change the plainttext for certain values of plaintetxt. Violates integrity (of ciphertexts). (e.g. this does not MAC the IV)
  - One can often then also learn information about the plaintext
- Even if the SKES is secure and the MAC is secure, the MAC then encrypt combination might be insecure

♣

Definition 3.1.7 (First Class Primitives for Authentication Encryption)

The fastest way to achieve authenticated encryption is to use a block cipher mode of operation with authentication built in:

- CCM mode (counter with CBC MAC)
- EAX mode
- GCM mode
- OCB mode

The modes of operation require a block cipher, but not a separate MAC (authentication functionality is built in)

♣

Definition 3.1.8 (Authenticated Encryption with Associated Data)

Common Scenario: Parts of a message must remain unencrypted, but still remain authenticated

Unencrypted data that travels with ciphertext = associated data

♣

Definition 3.1.9 (Authenticated Encryption with Associated Data (AHEAD))

Goals: We have a message m and some associated data d. We want to produce a cipher text c and a tag t, such that:

- The ciphertext provides semantic security for the message m
- The tag t authenticates the message m and the associated d
- All of this is implemented in a dummy proof system

To avoid combining secure encryption schemes and MACS and still getting and still getting insecure authenticated encryption schemes, we want non-experts to have a single function that does both

♣

3.2 Chosen Ciphertext Attacks

Definition 3.2.1 (Chosen Ciphertext Attacks)

One of our secondary goals in authenticated encryption is to prevent chosen ciphertext attacks.

Our goal is:

IND-CPA secure encryption + unforgeable MAC $\rightarrow$ IND-CCA Secure encryption

♣

Note

If we include a MAC of the ciphertext, then the adversary cannot modify the ciphertext without causing the MAC verification to fail.

When the MAC verification fails, the adversary learns nothing from the decryption oracle.

Thus, the adversary can only ask for decryption of ciphertexts it has previously encrypted – and the decryption oracle is useless.

We can formalize this with a more sophisticated interactive security reduction proof.

3.3 Pseudorandom Functions

Definition 3.3.1 (Pseudorandom Generator)

A pseudorandom generator is a deterministic function that takes as input a uniform seed $k \in \{0,1\}^l$ and outputs a random looking binary string of length l

$$PRG : \{0,1\}^l \rightarrow \{0,1\}^l$$

♣

Definition 3.3.2 (Pseudorandom Function)

A pseudorandom function is a determinism function that takes as input a uniform random seed $k \in \{0,1\}^l$ and a (non secret) label in $\{0,1\}^*$ and outputs a random looking binary string of length l

$$PRF : \{0,1\}^l \times \{0,1\}^* \rightarrow \{0,1\}^l$$

♣

Definition 3.3.3 (Security Property for PRGs and PRFs)

Indistinguishability:

Assuming the seed is uniformly random on $\{0,1\}^l$, it should be computationally infeasible for an adversary to distinguish the output of a PRG / PRF from a uniformly random string

For PRGs, the adversary gets either the real output of PRG under an unknown seed, or a random output, and must decide which

For PRFs, the adversary can make many calls to an oracle where the adversary can supply a label, and either always get the real output of the PRF using the same unknown seed applied to the label, or always gets a randomly chosen output (for distinct labels), and must decide which.

PRGs and PRFs assume that the random seed is a truly random (uniform) secret

Definition 3.3.4 (Key Derivation Function)

A key derivation function is a deterministic function that takes as input a (not necessarily uniform) random seed $k \in \{0,1\}^l$ and a (non-secret) label in $\{0,1\}^*$ and outputs a random looking binary string of length l

$$KDF : \{0,1\}^l \times \{0,1\}^* \to \{0,1\}^l$$

Remark

Difference between KDFs and PRFs:

KDF output should be indistinguishable from random even if the key k is non-uniform but sufficiently high entropy

Corollary 3.3.4.1 (Uses of PRGs, PRFs, KDFs)

PRGs: Expanding a strong uniform short key into a long pseudorandom key (e.g. stream cipher)

PRFs: Deriving many pseudorandom keys from a single short uniform key

KDFs: Turning longer non-uniform keys into shorter uniform-ish keys

3.4 Password Hashing

Definition 3.4.1 (User Authentication)

- Authenticators can be categorized as:
  - Knowledge based
  - Object based
  - ID based
  - Location based
- Multi factor authentication uses combinations from multiple different categories of authenticators

♣

Definition 3.4.2 (Entropy)

- Entropy measure the uncertainty in values generated from a random process
- Think of passwords being generated from a random process with a certain distribution
- Predicts the number of guesses we have to make to learn the password
- Suppose a process X generates one of n values of $x_1, ..., x_n$ with probabilities $p_1, ..., p_n$
- Formula for entropy of process X:

$$H(X) = -\sum_{i=1}^{n} p_i \ log_2(p_i)$$

- Simple way of thinking about it:
  - If a password is chosen uniformly at random from a set of size 2^n
  - Then its entropy n bits
  - We require around 2^{n-1} guesses on average to find it
- If some words are more likely than other there's less uncertainty
  - $\rightarrow$ less entropy
  - $\rightarrow$ easier to guess
- Entropy of passwords is a combination of length of password and randomness of each part of the password

♣

Definition 3.4.3 (Storing Passwords Securely)

1. Storing plaintext passwords in database
   - Problem: You're cooked
2. Storing encrypted version of password in database
   - Problem: If someone learns the key, they can decrypt the whole database
3. Store password using an irreversible transformation (hash)
   - Problem: None really- if someone forgets their password you cant recover it

♣

Corollary 3.4.3.1 (Password Hash Cracking)

- Attacking using Brute Force
  - Slow and expensive
- Attacking using Hash Tables
  - More difficult to crack one password has, but can reuse work (pre computation)
  - Requires massive amount of storage
- Attacking using Rainbow Tables (not on midterm hooray)
  - omitting for now

♡

Definition 3.4.4 (Salting)

- **Registration:**
  - Pick a random $\geq$ 80-bit salt
  - Store username, salt, and $H(\text{password} \parallel \text{salt})$ in database where H is a cryptographic hash function
- **Login:**
  - User supplies username and purported password
  - Look up username, salt, and hash in database
  - Check if $H(\text{password} \parallel \text{salt}) = \text{stored hash}$

♣

Corollary 3.4.4.1 (Benefits of Salting)

- Salting protects against precomputed hash tables / rainbow tables since you would need a different table for each salt
- Salting doesn't make it harder to do a brute force search against a single hash
- Salting does make brute force attacks against many hashes harder because you can't reuse the work from one attack on another attach

♡

Definition 3.4.5 (Password Hardening)

- You can slow down brute force attacks even more by hashing the password multiple times
- Doesn't slow login much, but slows brute force by a factor of the number of times you hashed

♣